

Do Sheaf Neural Networks Use Holonomy? A Measure–Intervene–Control Study

Ankit Grover and Rémi Bourgerie

KTH Royal Institute of Technology, Stockholm, Sweden
`{agrover, remibo}@kth.se`

Abstract. Geometric architectures are often motivated by internal mechanisms, but accuracy alone does not show whether predictions use them. In Sheaf Neural Networks (SNNs), edge transports form a connection whose cycle products define holonomy. We ask whether training changes triangle holonomy, whether predictions rely on the learned connection, and whether holonomy drives triangle counting. We use basis-independent loop readouts with identity interventions and shortcut controls. On high-homophily GRAPHUNIVERSE graphs, triangle counting increases the mean $SO(2)$ triangle rotation in Neural Sheaf Propagation (NSP) from 0.010 to 0.388 radians, while community detection ends at 0.029 radians. With more data, learned $SO(2)$ –NSP outperforms Identity NSP, and replacing its transports after training increases error further. However, ridge regression is more accurate, diagonal maps improve without continuous rotation, and fixed-degree models develop rotation without improved counting. Thus NSP can learn and rely on a nontrivial connection, but our experiments do not show that triangle holonomy drives its predictions.

Keywords: Geometric deep learning · Sheaf Neural Networks · Cellular sheaves · Geometric inductive biases · Holonomy · Interpretability

1 Introduction

Geometric architectures are often interpreted through the rules they encode. Architectural guarantees, however, are not mechanistic explanations. An $SE(3)$ -equivariant pose estimator or physics-aware scene-flow model may satisfy a geometric constraint by construction. Accuracy alone cannot reveal its driving cues. Interpretability work makes an analogous distinction between detecting an internal property and testing its behavioral use through a removal intervention [6]. Reliable geometric learning therefore needs mechanism-level tests, not only benchmark scores.

Sheaf Neural Networks equip a graph with additional local geometry: vector spaces, or *stalks*, on nodes and edges, linked by learned restriction maps at each incidence [3, 9, 10]. These maps induce transports that carry vectors between neighboring node stalks. The collection of transports forms a discrete connection, and their product around a cycle is its holonomy [2]. Triangles are a natural test because message-passing GNNs have known counting limitations and they are the shortest cycles on which a learned connection can exhibit holonomy [5, 13].

Prior work finds that non-identity sheaf maps can be dispensable at task level [11] and separates this from fixed-checkpoint reliance [12], but does not examine their cycle products. Sevetlidis and Pavlidis estimate holonomy by aligning embeddings along closed paths of nearby inputs; we instead compose transports explicitly defined on graph edges [14]. Higher-order networks take a different route: they represent each triangle explicitly as its own rank-2 cell, rather than leaving it implicit as a three-edge cycle [8]. Whether graph-based SNN predictions use their learned triangle-loop geometry therefore remains unresolved.

We ask three questions: How do triangle-loop products change during training across tasks? Do predictions rely on the complete learned connection at a fixed checkpoint? After controlling for predictive shortcuts, is there evidence that triangle holonomy is used for triangle counting? Our experiments use GRAPHUNIVERSE, a synthetic graph generator based on a degree-corrected stochastic block model (DC-SBM). In this graph family, triangle counts can covary with graph size, degree heterogeneity, and community structure [18]. Models may exploit these predictive shortcuts [7]. We therefore compare three NSP connection conditions and two non-neural models. We also train an SCCN with explicit triangle simplices and generate fixed-degree graphs.

Using TopoBench [17], we *measure* trained triangle-loop products, *intervene* on the complete learned connection at a fixed checkpoint, and use *controls* to assess evidence for triangle-specific computation.

2 Loop Measurement and Experimental Design

Each layer learns feature-conditioned restriction maps whose products give transports $T_{u \rightarrow v}^{(\ell)}$. For a cycle $C = (v_0, \dots, v_{k-1}, v_0)$, a local vector $x \in \mathcal{F}(v_0) \simeq \mathbb{R}^2$ is acted on by

$$x \mapsto H_C^{(\ell)} x, \quad H_C^{(\ell)} = T_{v_{k-1} \rightarrow v_0}^{(\ell)} \cdots T_{v_0 \rightarrow v_1}^{(\ell)}. \quad (1)$$

Figure 1 illustrates the closed-loop action. We measure this operator, not one observed activation. All readouts use the final-layer loop product $H_C^{(2)}$, computed from layer-1 representations. Initialization and trained measurements therefore use the same layer (Appendix A.1). Under independent orthonormal changes of stalk basis, $H_C^{(2)}$ changes only by conjugation. The readouts are therefore basis independent (Appendix A.3). For $H = QP$ in $d = 2$, the readouts are $\text{twist}(H) = \arccos(\text{tr } Q/2)$, $\text{gain}(H) = |\det H|$, and $\text{flip}(H) = \text{sign}(\det H)$. These measure rotation magnitude, stalk-space area scaling, and orientation reversal.

All three readouts are needed. $\text{SO}(2)$ maps use rotation while fixing gain to one and flip to +1. Diagonal maps lack continuous rotation but can change scaling and orientation. General maps combine rotation with anisotropic scaling.

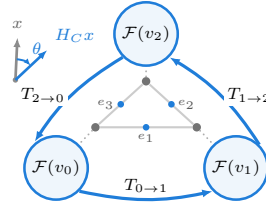


Fig. 1: Discrete triangle holonomy. Edge stalks (blue) receive maps from incident node stalks. Loop transport sends x to $H_C x$, producing mismatch θ .

Models. We use two-layer Neural Sheaf Propagation (NSP) as the primary model [16] and repeat the triangle-loop measurements with Neural Sheaf Diffusion (NSD) [3]. As an explicit-triangle control, we train a two-layer Simplicial Complex Convolutional Network (SCCN) [19] on clique-complex lifts, where every graph triangle is a rank-2 simplex (Appendix B.7). Sheaf models use $d = 2$ with diagonal, $\text{SO}(2)$, or general restrictions.

Connection control and intervention. Learned $\text{SO}(2)$ -NSP trains its transports jointly with all other weights and serves as the reference. Identity NSP is the training-time control: it fixes every transport to I_d while training the remaining weights. Comparing their test errors asks whether learning non-identity transports helps. Identity-replaced NSP is the test-time intervention: it replaces the learned transports with I_d while leaving all other weights unchanged. The error change asks whether the trained model relies on its transports. Because this intervention replaces the complete connection, it does not isolate triangle holonomy (Appendix B.6).

Tasks, regimes, and evaluation. On GRAPHUNIVERSE, we compare triangle counting with community detection, where node features and homophilous neighborhoods provide predictive cues that do not depend on triangle loops. This comparison uses 30 training and 8 test graphs across 10 splits. A three-seed sweep repeats the connection comparisons with 30–3000 training graphs. A fixed-degree control uses random 6-regular graphs, holding graph size and every node degree constant while triangle counts vary (Appendix B.3). For evaluation, triangle counting uses test MSE on the standardized $\log(1 + y)$ target computed from training statistics; community detection uses accuracy. Ridge regression tests graph-size and degree shortcuts without access to adjacency or enumerated triangles (Appendix B.5, Eq. 14). We also report $\Delta_{\text{const}} = \text{MSE}_{\text{model}} - \text{MSE}_{\text{mean}}$, where negative values indicate improvement.

3 Results

Training on triangle counting changes the $\text{SO}(2)$ loop products. For NSP, triangle counting increases triangle-weighted mean loop rotation from 0.010 to 0.388 ± 0.078 rad (Appendix A.4). The community-detection comparison ends at 0.029 ± 0.005 rad (Fig. 2a–b). The increase is similar for NSP and NSD: 0.378 ± 0.078 and 0.405 ± 0.075 rad. For community detection, seven NSD runs stay near identity. The remaining three show high rotation and lower accuracy. We therefore use NSP for the main task comparison and report the NSD results descriptively (Appendix C.1).

The setting admits multiple predictive routes. At $N_{\text{train}} = 30$, ridge regression and SCCN reach MSE 0.110 and 0.121. The best graph-based SNN reaches 0.71. The sampled GRAPHUNIVERSE regime therefore contains shortcuts based on graph size and degree statistics. It also rewards explicit access to triangles. This does not show that the SNNs use either source.

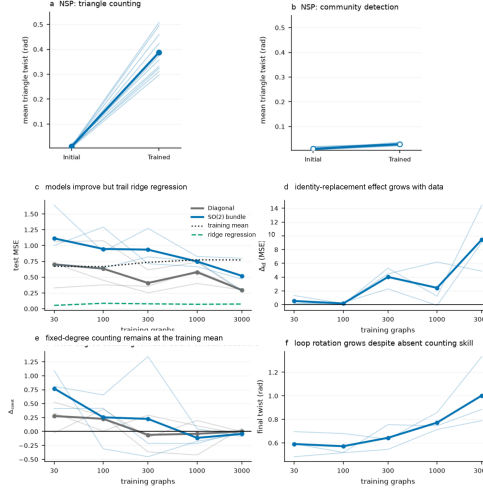


Fig. 2: Measure–intervene–control results in NSP. Thin lines show runs and thick lines show means. Panels (c–f) use three seeds. (a–b) Counting increases final-layer twist, while community detection remains near identity. With more data, (c) MSE falls but remains above ridge regression, and (d) the identity-replacement effect grows. On random 6-regular graphs, (e) performance remains near the training mean, while (f) twist approaches one radian.

Learning and retaining the connection both matter. Learned $SO(2)$ –NSP has lower mean MSE than Identity NSP throughout the sweep. The separation is clear from $N_{\text{train}} = 300$ onward. Replacing the learned connection after training raises the error further. The same pattern appears on 200 new test graphs. Identity NSP tests whether learning the connection helps. Identity-replaced NSP tests whether a trained checkpoint relies on it. Neither result isolates triangle holonomy (Fig. 2c–d and Appendix D.2–D.3).

Increasing rotation does not establish triangle-specific computation. SCCN has lower mean MSE than each NSP condition (Appendix E.3). Ridge regression is also highly accurate. Diagonal maps improve even though they cannot express continuous rotations. On fixed-degree graphs, bundle rotation approaches one radian, but performance remains near the training mean (Fig. 2e–f and Appendix E.2). These results show that rotation is not sufficient for better counting. They do not identify rotation as the mechanism used by learned $SO(2)$ –NSP.

4 Conclusion

In the larger-data regime, learned connections have training-time value and fixed-checkpoint reliance. However, ridge regression, diagonal maps, and fixed-degree graphs provide no evidence that triangle holonomy drives predictions. Our basis-independent loop readouts may provide a useful template for testing geometric biases. Limitations appear in Appendix F.

References

1. Abboud, R., Ceylan, İ.İ., Grohe, M., Lukasiewicz, T.: The surprising power of graph neural networks with random node initialization. In: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI) (2021)
2. Barbero, F., Bodnar, C., Sáez de Ocáriz Borde, H., Bronstein, M., Veličković, P., Liò, P.: Sheaf neural networks with connection laplacians. In: Proceedings of Topological, Algebraic, and Geometric Learning Workshops 2022. Proceedings of Machine Learning Research, vol. 196, pp. 28–36. PMLR (2022)
3. Bodnar, C., Di Giovanni, F., Chamberlain, B.P., Liò, P., Bronstein, M.M.: Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in GNNs. In: Advances in Neural Information Processing Systems (2022)
4. Cartwright, D., Harary, F.: Structural balance: a generalization of Heider’s theory. *Psychological Review* **63**(5), 277–293 (1956)
5. Chen, Z., Chen, L., Villar, S., Bruna, J.: Can graph neural networks count sub-structures? In: Advances in Neural Information Processing Systems (2020)
6. Elazar, Y., Ravfogel, S., Jacovi, A., Goldberg, Y.: Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics* **9**, 160–175 (03 2021). https://doi.org/10.1162/tac1_a_00359, https://doi.org/10.1162/tac1_a_00359
7. Geirhos, R., Jacobsen, J.H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F.A.: Shortcut learning in deep neural networks. *Nature Machine Intelligence* **2**(11), 665–673 (2020). <https://doi.org/10.1038/s42256-020-00257-z>
8. Hajij, M., Bastian, L., Osentoski, S., Kabaria, H., Davenport, J.L., Dawood, S., Cherukuri, B., Kocheemoolayil, J.G., Shahmansouri, N., Lew, A., Papamarkou, T., Birdal, T.: Copresheaf topological neural networks: A generalized deep learning framework. In: The Thirty-ninth Annual Conference on Neural Information Processing Systems (2025), <https://openreview.net/forum?id=3G56xC1PYg>
9. Hansen, J., Gebhart, T.: Sheaf neural networks. In: NeurIPS Workshop on Topological Data Analysis and Beyond (2020)
10. Hansen, J., Ghrist, R.: Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology* **3**, 315–358 (2019)
11. Hernandez Caralt, F., González i Català, M., Bazaga, A., Liò, P.: On the necessity of learnable sheaf Laplacians. In: ICLR 2026 Workshop on Geometry-grounded Representation Learning and Generative Modeling (GRaM), Tiny Paper Track (2026), arXiv:2603.05395
12. Liu, Y.: Learned, relied upon, or necessary? separating checkpoint dependence from task-level value in sheaf GNNs. arXiv preprint arXiv:2607.25387 (2026). <https://doi.org/10.48550/arXiv.2607.25387>
13. Morris, C., Ritzert, M., Fey, M., Hamilton, W.L., Lenssen, J.E., Rattan, G., Grohe, M.: Weisfeiler and Leman go neural: Higher-order graph neural networks. In: Proceedings of the AAAI Conference on Artificial Intelligence (2019)
14. Sevetlidis, V., Pavlidis, G.: Gauge-invariant representation holonomy. In: The Fourteenth International Conference on Learning Representations (2026), <https://openreview.net/forum?id=czJqKToDGq>
15. Singer, A., Wu, H.T.: Vector diffusion maps and the connection Laplacian. *Communications on Pure and Applied Mathematics* **65**(8), 1067–1144 (2012)
16. Suk, J., Giusti, L., Hemo, T., Lopez, M., Barnmpas, K., Bodnar, C.: Surfing on the neural sheaf. In: NeurIPS 2022 Workshop on Symmetry and Geometry in Neural Representations (2022)

17. Telyatnikov, L., Bernardez, G., Montagna, M., Hajij, M., Carrasco, M., Vasylenko, P., Papillon, M., Zamzmi, G., Schaub, M.T., Verhellen, J., Snopov, P., Miquel-Oliver, B., Gil-Sorribes, M., Molina, A., Guallar, V., Long, T., Suk, J., Rygiel, P., Nikitin, A.V., Escalona, G., Banf, M., Filipiak, D., Imasheva, L., Schattauer, M., Martinez, A.L., Fritze, H., Masden, M., Sánchez, V., Lecha, M., Cavallo, A., Battiloro, C., Piekenbrock, M., Tec, M., Dasoulas, G., Miolane, N., Scardapane, S., Papamarkou, T.: TopoBench: A framework for benchmarking topological deep learning. *Journal of Data-centric Machine Learning Research* (2025)
18. Van Langendonck, L., Bernárdez, G., Miolane, N., Barlet-Ros, P.: GraphUniverse: Synthetic graph generation for evaluating inductive generalization. In: *International Conference on Learning Representations* (2026), arXiv:2509.21097
19. Yang, R., Sala, F., Bogdan, P.: Efficient representation learning for higher-order data with simplicial complexes. In: Rieck, B., Pascanu, R. (eds.) *Proceedings of the First Learning on Graphs Conference*. *Proceedings of Machine Learning Research*, vol. 198, pp. 13:1–13:21. PMLR (2022)

Appendix Contents

A	Loop-Product Measurement and Validation	8
A.1	Edge Transports and Loop Products	8
A.2	Twist, Gain, and Flip	8
A.3	Basis Independence	9
A.4	Aggregation over Test Triangles	9
A.5	Local Triangle Loops and Global Connection Consistency	10
A.6	Validation of Measurement and Identity Replacement	10
B	Data, Models, and Evaluation Protocol	10
B.1	Graph Regimes and Tasks	10
B.2	Generating the GRAPHUNIVERSE Graphs	11
B.3	Generating the Fixed-Degree Graphs	12
B.4	Architectures, Pooling, and Optimization	12
B.5	Targets, Metrics, and Non-Neural Models	12
B.6	Learned, Identity, and Identity-Replaced NSP	13
B.7	Simplicial Complex Convolutional Network	14
	Results at a Glance	14
C	How Training Changes Loop Geometry	15
C.1	NSP and NSD Across Tasks	15
C.2	How Restriction Families Change the Readouts	15
D	Does Performance Depend on the Learned Connection?	16
D.1	Identity Replacement with 30 Training Graphs	16
D.2	Performance across Training-Set Sizes	17
D.3	Robustness on 200 Test Graphs	17
E	Does Loop Rotation Explain Triangle Counting?	18
E.1	Loop Rotation versus Counting Error	18
E.2	Results on Generated Fixed-Degree Graphs	19
E.3	Comparison with SCCN	19
F	Limitations	19
	Acknowledgements	20

A Loop-Product Measurement and Validation

A.1 Edge Transports and Loop Products

Let $G = (V, E)$ be an undirected graph. A cellular sheaf assigns a stalk $\mathcal{F}(v) \simeq \mathbb{R}^d$ to every node. It also assigns a restriction map $\mathcal{F}_{v \leq e}^{(\ell)} : \mathcal{F}(v) \rightarrow \mathcal{F}(e)$ to each incident node–edge pair at propagation layer ℓ . For an edge $e = \{u, v\}$, the unnormalized off-diagonal block and derived transport are

$$(L_{\mathcal{F}}^{(\ell)})_{vu} = -(\mathcal{F}_{v \leq e}^{(\ell)})^\top \mathcal{F}_{u \leq e}^{(\ell)}, \quad T_{u \rightarrow v}^{(\ell)} = (\mathcal{F}_{v \leq e}^{(\ell)})^\top \mathcal{F}_{u \leq e}^{(\ell)} = -(L_{\mathcal{F}}^{(\ell)})_{vu}. \quad (2)$$

We reserve $\Delta_{\mathcal{F}}^{(\ell)} = (D_{\mathcal{F}}^{(\ell)})^{-1/2} L_{\mathcal{F}}^{(\ell)} (D_{\mathcal{F}}^{(\ell)})^{-1/2}$ for the normalized Laplacian. For orthogonal restriction maps, transpose equals inverse and the transports form a discrete $O(d)$ connection. TopoBench’s two-dimensional bundle parameterization uses the Cayley transform and produces proper rotations in $SO(2)$.

For a cycle $C = (v_0, v_1, \dots, v_{k-1}, v_0)$, the layer- ℓ loop product based at v_0 is

$$H_C^{(\ell)} = T_{v_{k-1} \rightarrow v_0}^{(\ell)} \cdots T_{v_1 \rightarrow v_2}^{(\ell)} T_{v_0 \rightarrow v_1}^{(\ell)}. \quad (3)$$

It acts on $x \in \mathcal{F}(v_0)$ by $x \mapsto H_C^{(\ell)} x$. The measurement records the linear operator induced by the learned connection. It does not trace one observed hidden activation around the graph. Each model layer has its own feature-conditioned sheaf learner. All reported readouts use the final-layer product $H_C^{(2)}$. Its transports are computed from representations already processed by layer 1. Initialization and trained-model measurements both use layer 2. Identity replacement changes transports in both layers. It therefore tests the complete learned connection rather than only the measured final layer.

For isometric transports, $H_C^{(\ell)}$ is the holonomy of the discrete connection. For diagonal and general restrictions, we use the more neutral term *loop product*.

A.2 Twist, Gain, and Flip

Let $H = QP$ be the polar decomposition, with Q orthogonal and $P \succeq 0$. Because all experiments use $d = 2$, we compute

$$\begin{aligned} \text{twist}(H) &= \arccos\left(\frac{\text{tr}(Q)}{2}\right), \\ \text{gain}(H) &= |\det H|, \quad \text{flip}(H) = \text{sign}(\det H). \end{aligned} \quad (4)$$

The main text describes these as loop-rotation magnitude, area factor, and orientation sign. In general dimension, the determinant is a volume multiplier. Here $|\det H|$ specifically multiplies two-dimensional stalk-space area. Gain below one means that the loop product contracts area. It does not mean that the graph shrinks. It also does not require every vector norm to decrease. One singular direction may expand while the other contracts more strongly.

The interpretation depends on the restriction family.

- **SO(2) bundle maps.** $P = I_d$, gain is exactly one, flip is +1, and twist is the magnitude of the ordinary rotation angle in $[0, \pi]$. This is the setting used for the central rotation claims.
- **Diagonal maps.** Loop products remain diagonal. They have no continuous rotation degree of freedom. The orientation sign is the product of edge signs and corresponds to $\mathbb{Z}_2$ frustration in signed-graph balance theory [4].
- **General maps.** The polar factor separates rotation from net area change, but its rotation estimate becomes unstable when H is nearly singular.

These differences are why we do not combine the three readouts into one generic geometry score.

A.3 Basis Independence

Let $G_v \in O(d)$ be an independent orthogonal change of coordinates in each node stalk. Restrictions and transports transform as

$$\mathcal{F}_{v \leq e}^{(\ell)} \mapsto \mathcal{F}_{v \leq e}^{(\ell)} G_v^\top, \quad T_{u \rightarrow v}^{(\ell)} \mapsto G_v T_{u \rightarrow v}^{(\ell)} G_u^\top. \quad (5)$$

Substitution into the loop product gives

$$(H_C^{(\ell)})' = (G_{v_0} T_{v_{k-1} \rightarrow v_0}^{(\ell)} G_{v_{k-1}}^\top) \cdots (G_{v_1} T_{v_0 \rightarrow v_1}^{(\ell)} G_{v_0}^\top) \quad (6)$$

$$= G_{v_0} H_C^{(\ell)} G_{v_0}^\top, \quad (7)$$

because every interior factor $G_v^\top G_v$ cancels. Thus a loop product changes by conjugation only. Gain and flip are invariant under orthogonal conjugation. For nonsingular H , the polar factor is unique and transforms as $Q \mapsto G Q G^\top$, so twist is invariant as well. The reported quantities therefore do not depend on the orthonormal coordinate bases chosen in the stalks.

A.4 Aggregation over Test Triangles

All reported loop summaries are triangle-weighted. If $\mathcal{C}_3(G)$ is the set of triangles in test graph G , the reported mean rotation is

$$\bar{\theta}_{\text{tri}} = \frac{\sum_{G \in \mathcal{D}_{\text{test}}} \sum_{C \in \mathcal{C}_3(G)} \text{twist}(H_C^{(2)})}{\sum_{G \in \mathcal{D}_{\text{test}}} |\mathcal{C}_3(G)|}. \quad (8)$$

Every enumerated triangle contributes equally. Graphs with more triangles therefore contribute more loop observations. Per-graph loop means were not retained. We therefore do not report graph-balanced reweighting. This convention affects the interpretation of the mean. It does not affect the layer, basis-independence, or identity-replacement definitions.

A.5 Local Triangle Loops and Global Connection Consistency

For $\text{SO}(2)$ transports, global consistency means that a coordinate frame can be assigned to every node. The relative frames at an edge must explain its transport. Such a connection has identity transport around every closed cycle. A nonzero triangle-loop rotation is therefore a local certificate of inconsistency on that triangle.

Singer and Wu use the connection Laplacian to study graph-wide synchronization and frustration [15]. In contrast, we compose learned transports around individual triangles and measure their local loop products. Our measurement does not estimate global spectral frustration or consistency on a longer cycle basis. The two views are complementary rather than interchangeable.

A.6 Validation of Measurement and Identity Replacement

Section A.3 proves the invariance property mathematically. The checks below test its numerical implementation. They also verify that bundle loop products remain in $\text{SO}(2)$. A perturbation test measures the stability of bundle rotation. A final check confirms that identity replacement changes the intended transports. These tests validate the measurement and intervention code. They are not additional learning experiments. Reporting them makes the procedure reproducible for future studies that use the same readouts.

Table 1: Numerical checks on the loop measurement and identity-replacement implementation.

Check	What it establishes	Result
Basis change	The code agrees with the conjugation-invariance derivation.	Maximum readout change 2.8×10^{-15} across 1,000 random orthogonal basis changes.
$\text{SO}(2)$ constraint	Cayley-parameterized transports and their loop products remain proper rotations.	$ \det H = 1.000$ for measured bundle loops.
Perturbation stability	Bundle twist is numerically stable at the scale used in the experiments.	Perturbations of size 10^{-4} change rotation by 6.86×10^{-5} rad.
Nearly singular maps	The same rotation readout is unreliable for ill-conditioned general loop products.	The perturbation changes general-map rotation by 0.195 rad in the near-singular case.
Identity replacement	Both transport layers are replaced as intended and the resulting bundle loops are identities.	$H_C^{(2)} = I_d$: twist 0, gain 1, and zero Frobenius error.

B Data, Models, and Evaluation Protocol

B.1 Graph Regimes and Tasks

The main experiments use a custom high-homophily GRAPHUNIVERSE regime rather than the benchmark’s standard regime grid. The average-degree range

is set to 5–7 so that sampled graphs contain enough triangles for repeated loop measurements. Community detection is a comparison task under the same generator. Node features and homophilous neighborhoods provide direct predictive cues. They do not require closed-loop transport. This task is not a cycle-isolating control. The conclusions remain scoped to this synthetic regime.

The second graph regime uses generated random 6-regular graphs. It complements GRAPHUNIVERSE by holding graph size, edge count, and the complete degree sequence fixed while wiring and triangle count vary. This removes the graph-size and degree cues available to ridge regression in the main regime.

B.2 Generating the GRAPHUNIVERSE Graphs

GRAPHUNIVERSE uses a two-stage degree-corrected stochastic block model (DC-SBM).

Fixed universe. A master universe is created once with seed 42. It contains $K = 20$ universe-level communities and a 20×20 edge-propensity matrix with variance 1.0. Node features form 15-dimensional Gaussian clusters. Community centers have variance 0.2. Within-community features have variance 0.4.

Graph-family sampling. For each run, graph-level parameters are sampled from the ranges in Table 2. Conditional on community labels c_u, c_v and Pareto degree factors θ_u, θ_v , edge probabilities are proportional to

$$B_{c_u c_v} \theta_u \theta_v, \quad (9)$$

then clipped to $[0, 1]$. Graphs are made undirected and connectivity is enforced. Data generation and model initialization are independently resampled across seeds. A data split consists of one generated training set and one separately generated test set associated with seed s . All methods compared within a split receive exactly those graphs and the same transformed target.

Table 2: GRAPHUNIVERSE settings. Ranges are sampled independently for each graph family.

Setting	Value
Nodes per graph	100–160
Communities per graph	5–7, drawn from 20 universe communities
Average degree	5.0–7.0
Power-law exponent	2.0–2.5
Degree separation	0.5–1.0
Homophily	$[0.9, 1.0]$
Main split	30 training and 8 test graphs per seed
Larger test	200 newly generated test graphs for selected settings
Training-set sizes	$N_{\text{train}} \in \{30, 100, 300, 1000, 3000\}$, with the same 8 test graphs within each seed

B.3 Generating the Fixed-Degree Graphs

The fixed-degree graphs are generated separately from GRAPHUNIVERSE using NetworkX random regular graphs,

$$G_i^{(s)} \sim \text{random_regular_graph}(r = 6, n = 100, \text{seed} = 1000s + i). \quad (10)$$

Every graph has 100 nodes, 300 edges, and degree six at every node. Only its wiring and triangle count vary. We evaluate two feature conditions. Independent 15-dimensional Gaussian features act as symmetry breakers. Constant all-ones features retain anonymous message passing. The split construction and standardized log-count target in Eq. 12 otherwise match the GRAPHUNIVERSE experiment.

B.4 Architectures, Pooling, and Optimization

Table 3: Architecture and training settings.

Setting	Value
Neural architectures	NSP [16], NSD [3], and SCCN [19]
Sheaf restrictions	diagonal, $\text{SO}(2)$ bundle, and general 2×2
Sheaf model size	2 layers, stalk dimension $d = 2$, and feature width 32
Graph-level readout	mean pooling followed by an MLP
Loop measurement	final-layer product $H_C^{(2)}$
Optimization	Adam with learning rate 10^{-2} , batch size 8, and 60 epochs
Main task comparison	30 training graphs and 2 sheaf dynamics $\times$ 2 tasks $\times$ 3 restriction families $\times$ 10 seeds
Training-size sweep	$N_{\text{train}} = 30, 100, 300, 1000, 3000$, with 3 seeds and fixed test graphs within each seed

For graph-level triangle prediction, mean pooling keeps the representation scale from growing directly with graph size or simplex multiplicity. This is compatible with the standardized $\log(1 + y)$ target, which grows much more slowly than the raw count. We report this as the chosen parameterization, not as a separately evaluated pooling result.

B.5 Targets, Metrics, and Non-Neural Models

Community detection predicts the universe-level community identifier for each node, with labels in $\{0, \dots, 19\}$. Triangle counting first computes the raw graph-level count

$$y(G) = \frac{1}{3} \sum_{v \in V} \text{triangles}(v), \quad (11)$$

where each triangle is counted at its three vertices. For split s , the regression target is

$$z_s(G) = \frac{\log(1 + y(G)) - \mu_s}{\sigma_s}, \quad (12)$$

where μ_s and σ_s are computed from the training graphs only. They are the mean and standard deviation of $\log(1 + y)$. The raw counts span a wide, right-skewed range. The natural-log transform compresses that range. This prevents triangle-heavy graphs from dominating squared error and reduces target-scale variation. Adding one keeps the target defined when $y = 0$. Standardization gives the training target zero mean and unit variance within each split. All reported counting MSEs are errors in standardized log-count space. They are not absolute errors in raw triangle counts. Test graphs never contribute to the transformation.

The two non-neural models use the same training and test graphs and the same target z_s as the neural models.

The *training-mean model* ignores the input graph and always outputs $z_s = 0$. This is equivalent to predicting μ_s , the training-set mean of $\log(1 + y)$. It is not the mean of the raw triangle counts. Its test MSE is

$$\mathcal{L}_{\text{mean}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{G \in \mathcal{D}_{\text{test}}} z_s(G)^2. \quad (13)$$

It is computed separately for every split rather than assumed to equal one. The test graphs need not have the training variance. With eight test graphs, this value varies substantially. In the fixed-degree experiment it ranges from approximately 0.43 to 3.28 across seeds.

Ridge regression is an ℓ_2 -regularized linear model. It receives seven graph-level inputs. Two are node count and edge count. The other five are the minimum, maximum, mean, standard deviation, and median of the node degrees. For design matrix $Z \in \mathbb{R}^{m \times 7}$ and standardized targets $\mathbf{z} \in \mathbb{R}^m$, its parameters solve

$$(\hat{b}, \hat{\beta}) = \arg \min_{b, \beta} \|\mathbf{z} - b\mathbf{1} - Z\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad (14)$$

where b is an unpenalized intercept and $\lambda \geq 0$ controls regularization. Ridge regression never receives the adjacency matrix, node triples, or enumerated triangles. It asks whether the sampled GRAPHUNIVERSE family permits accurate prediction from graph size and the marginal degree distribution. Its success does not show that the SNNs use the same information.

B.6 Learned, Identity, and Identity-Replaced NSP

We compare three conditions built from the same two-layer NSP propagation architecture.

- **Learned SO(2)–NSP** jointly optimizes both layers’ Cayley-parameterized transports and all remaining model parameters.

- **Identity NSP** fixes every transport to I_d from initialization while optimizing all remaining NSP parameters under the same protocol. It is the NSP implementation of an Identity Sheaf Network [11].
- **Identity-replaced NSP** first trains learned $\text{SO}(2)$ -NSP. It then zeroes both layers’ Cayley parameters without retraining. Because $\text{Cayley}(0) = I_d$, every transport becomes I_d . All remaining weights retain the values learned with a non-identity connection.

Identity NSP asks whether allowing a non-identity connection improves the model obtained by training. Identity-replaced NSP instead asks whether a particular trained checkpoint relies on its learned connection. This distinction follows the separation between task-level value and checkpoint reliance [12]. The intervention is broad: it changes the complete two-layer connection and does not selectively remove triangle holonomy.

For the fixed-checkpoint intervention we report

$$\Delta_{\text{id}} = \mathcal{L}_{\text{identity-replaced}} - \mathcal{L}_{\text{learned}}. \quad (15)$$

Positive values mean that replacing the learned connection increases error.

B.7 Simplicial Complex Convolutional Network

We implement the two-layer Simplicial Complex Convolutional Network (SCCN) of Yang et al. [19]. Each input graph is lifted to its two-dimensional clique complex. Nodes are rank-0 simplices, and edges are rank-1 simplices. Every graph triangle becomes one rank-2 simplex. The model propagates across simplex ranks. It then mean-pools the learned rank-2 representations and passes the result through an MLP.

SCCN uses the same GRAPHUNIVERSE splits and standardized $\log(1 + y)$ target as the NSP training-size sweep. It also uses the same three seeds, 60 training epochs, and eight test graphs. Its representation is intentionally different from the graph-based SNNs. The clique-complex construction exposes every target triangle before learning. The SCCN result tests whether the protocol rewards explicit triangle simplices. It is not a representation-matched test of the sheaf connection.

Results at a Glance

Table 4 links each main conclusion to its central evidence and the appendix section containing the full result.

Table 4: Guide to the principal results.

Finding	Central evidence	Details
Counting changes bundle loop geometry	NSP twist increases from 0.010 to 0.388 ± 0.078 rad. Community detection ends at 0.029 ± 0.005 rad.	C.1
The complete learned connection has training-time value	Learned $\text{SO}(2)$ -NSP has lower mean MSE than Identity NSP, with clear separation from $N_{\text{train}} = 300$ onward.	D.2
Trained checkpoints rely on the learned connection	Identity-replaced NSP has higher mean MSE. The direction persists on 200 new test graphs.	D.2 , D.3
Accurate counting is possible in this protocol	SCCN reaches 0.070 ± 0.058 MSE at $N_{\text{train}} = 3000$ when triangles are explicit simplices.	E.3
Rotation alone does not explain counting	Rotation is weakly associated with error. Diagonal and fixed-degree results separate rotation from counting performance.	E.1 , E.2

C How Training Changes Loop Geometry

C.1 NSP and NSD Across Tasks

On triangle counting, the final-layer $\text{SO}(2)$ loop-rotation changes are similar for NSP and NSD. They are 0.378 ± 0.078 and 0.405 ± 0.075 rad, respectively. The difference appears in community detection. Seven of ten NSD runs stay near identity. The other three reach roughly 1.6 rad and have the lowest accuracies in that condition. We therefore report the NSD task comparison descriptively.

C.2 How Restriction Families Change the Readouts

Figure 3 shows the three measured responses. Bundle maps increase final-layer loop rotation while preserving stalk-space area exactly. Diagonal community-detection models reduce the fraction of loops with negative orientation sign, consistent with a more balanced sign pattern. Diagonal and general loop products contract strongly. A single scalar would hide these distinct behaviors and would make the unstable rotation estimate of nearly singular general maps appear more comparable than it is.

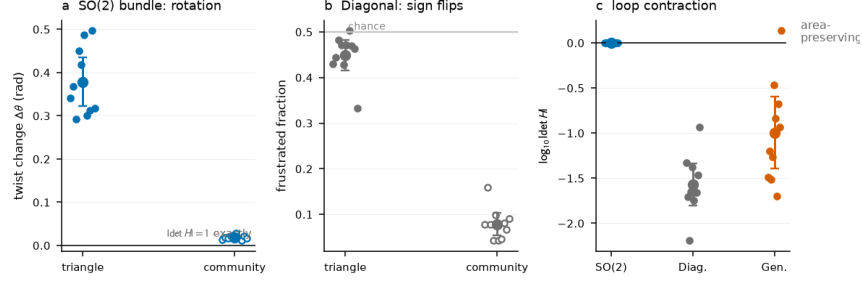


Fig. 3: Restriction families change different loop readouts (NSP). Triangle counting increases bundle rotation (a). Community detection reduces the fraction of diagonal loops with negative determinant below the 0.5 chance line (b). Only bundle loops preserve area. Diagonal and general loops contract it (c), where $\log_{10} |\det H| = 0$ denotes preservation. Points are seeds. Bars show mean 95% confidence intervals.

D Does Performance Depend on the Learned Connection?

D.1 Identity Replacement with 30 Training Graphs

For triangle counting, identity replacement increases MSE by 0.48 ± 0.68 for NSP and 0.17 ± 0.80 for NSD. Both 95% confidence intervals include zero. They also include increases large enough to matter. The small-data result is therefore unresolved. For community detection, the mean change is close to zero for both dynamics.

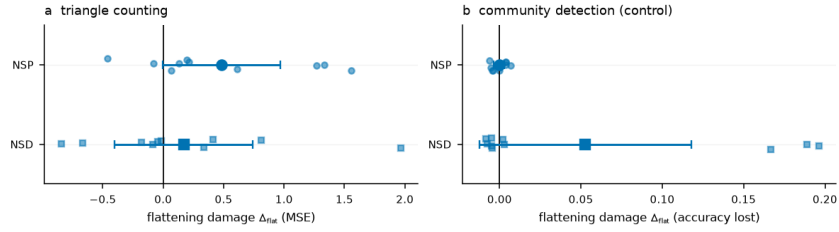


Fig. 4: Identity replacement is unresolved with 30 training graphs. Zero denotes no change. Positive values mean higher counting MSE (a) or lower community accuracy (b) after replacement. Counting seeds lie on both sides of zero, and both mean 95% confidence intervals cross it. Community effects are mostly near zero, with three larger NSD accuracy losses. Small marks are seeds. Large marks are means.

D.2 Performance across Training-Set Sizes

Table 5 separates three questions. These are training-time connection value, fixed-checkpoint reliance, and performance with explicit triangle simplices. Learned $\text{SO}(2)$ -NSP has lower mean MSE than Identity NSP at every training size. The first two differences are small relative to seed variability. From $N_{\text{train}} = 300$ onward, the mean separation is substantial. At $N_{\text{train}} = 3000$, learned $\text{SO}(2)$ -NSP reaches 0.521 ± 0.259 MSE. Identity NSP reaches 2.724 ± 0.549 .

Identity-replaced NSP has higher mean MSE than the learned model throughout the sweep. At $N_{\text{train}} = 3000$, it reaches 9.981 ± 4.652 . Thus the learned connection has value during training. The resulting checkpoints also depend on retaining it. Neither comparison identifies triangle holonomy as the causal mechanism. Both concern the complete connection.

Table 5: Performance across connection conditions and SCCN. Entries are standardized $\log(1 + y)$ test MSE. We report the mean $\pm$ sample standard deviation over three seeds. Lower is better. Identity replacement is applied after training without retraining.

N_{train}	Identity NSP	Learned $\text{SO}(2)$ -NSP	Identity-replaced NSP	SCCN
30	1.228 ± 0.265	1.114 ± 0.478	1.644 ± 1.167	0.121 ± 0.020
100	1.054 ± 0.584	0.946 ± 0.327	1.113 ± 0.207	0.144 ± 0.114
300	2.512 ± 1.607	0.935 ± 0.296	4.957 ± 1.610	0.091 ± 0.075
1000	2.949 ± 1.845	0.748 ± 0.080	3.185 ± 3.379	0.108 ± 0.047
3000	2.724 ± 0.549	0.521 ± 0.259	9.981 ± 4.652	0.070 ± 0.058

D.3 Robustness on 200 Test Graphs

The training-size curves use eight test graphs per seed. We selected the pivotal GRAPHUNIVERSE configurations at $N_{\text{train}} \in \{300, 3000\}$. We also selected the fixed-degree configuration at $N_{\text{train}} = 3000$. The original checkpoints were not saved. We therefore retrained each configuration with the same split, seed, and protocol. Each retrained model was evaluated on the original eight graphs and on 200 new test graphs. This is a larger-test robustness check, not an additional independent training seed.

The retrained eight-graph metrics closely match the originals. The median absolute difference is zero, and 72% of runs are within 0.01. A few $N_{\text{train}} = 3000$ runs differ across retraining. On the 200-graph tests, the mean identity-replacement effect remains positive at the selected GRAPHUNIVERSE settings. The fixed-degree models remain near the training-mean model. The larger test therefore supports the direction of the connection-reliance result. It does not remove the seed variability.

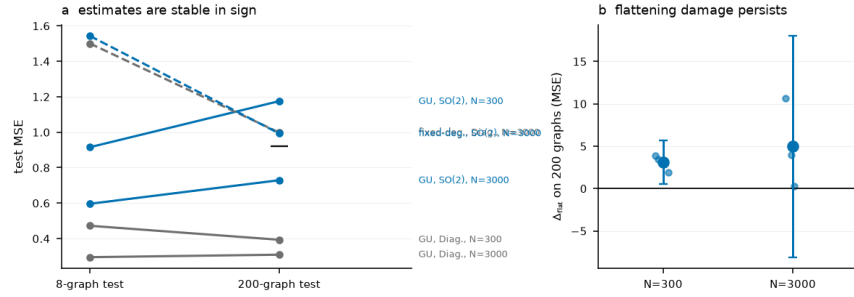


Fig. 5: Larger-test robustness check. Lines in (a) connect the same retrained model evaluated on 8 and 200 test graphs. They expose shifts caused by the small original test set. On 200 graphs, the mean identity-replacement effect remains positive at $N = 300$ and $N = 3000$ (b). The latter is highly variable. In (b), small marks are seeds. Large marks and bars are means with 95% confidence intervals.

E Does Loop Rotation Explain Triangle Counting?

E.1 Loop Rotation versus Counting Error

Within the $SO(2)$ bundle family, loop-rotation change is not negatively associated with triangle-counting error. Across the plotted runs, Pearson $r = -0.18$ with $p = 0.58$. Figure 6 plots error relative to the training-mean model. We do not pool restriction families in one regression. Family identity dominates that comparison, and continuous rotation is not meaningful for diagonal maps.

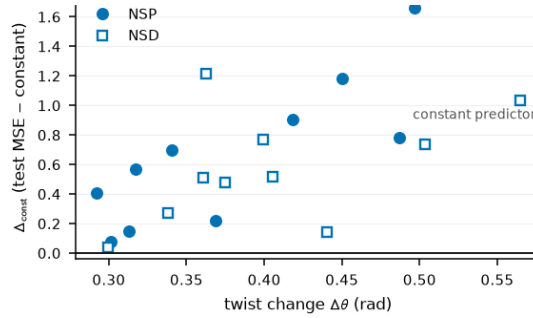


Fig. 6: More loop rotation is not associated with better counting. Each point is one seed. Circles denote NSP and squares denote NSD. The horizontal axis is the change in final-layer twist. The vertical axis is Δ_{const} , where lower values are better. The weak association ($r = -0.18$, $p = 0.58$) provides no evidence that larger rotation changes reduce error.

E.2 Results on Generated Fixed-Degree Graphs

On random 6-regular graphs with independent Gaussian node features, no restriction family consistently improves on the training-mean model. Bundle maps nevertheless develop substantial loop rotation. With constant node features, all families match the training-mean model to within $|\Delta| \leq 0.007$. The learned bundle transports remain identities, so identity replacement is vacuous. Random node features can make message passing more expressive than anonymous 1-WL [1]. The observed failure is therefore an empirical optimization and sample-efficiency result, not an expressivity theorem.

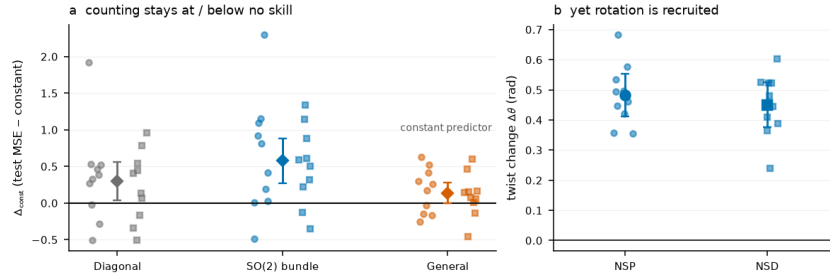


Fig. 7: Fixed degree separates loop rotation from counting skill. With Gaussian node features, no restriction family consistently falls below the $\Delta_{\text{const}} = 0$ training-mean reference in (a). Nevertheless, bundle twist increases for both NSP and NSD (b). Small marks are seeds. Large marks and bars are means with 95% confidence intervals.

E.3 Comparison with SCCN

SCCN has lower mean MSE than all three NSP conditions at every training size in Table 5. At $N_{\text{train}} = 3000$, it reaches 0.070 ± 0.058 . This shows that the protocol rewards explicit access to triangle simplices. It also addresses an alternative explanation: perhaps no trained model could learn the target under the chosen splits and transformed loss.

The result does not make SCCN a representation-matched baseline for the graph-based SNNs. The clique-complex lift enumerates every target triangle before learning. The SNNs receive only nodes and edges and must use edge-based propagation. SCCN therefore establishes learnability with explicit higher-order structure. It does not show that the SNNs should recover the same solution. It also does not show that their learned connection stores triangle counts.

F Limitations

The experiments cover synthetic high-homophily graphs, two-dimensional stalks, and triangle loops. The training-size study uses three seeds. The main task comparison relies on NSP because three NSD community-detection runs behave

differently from the other seven. Rotation from nearly singular general loop products is unreliable and is used only descriptively.

Identity replacement is a broad intervention. It changes both learned transport layers after training. This may disrupt weights that adapted to the original connection. Identity NSP reduces this concern. Learned $\text{SO}(2)$ -NSP also has lower mean error when the alternative is trained from initialization. Nevertheless, neither comparison reveals what information the complete connection stores. Neither isolates triangle holonomy. A matched-size connection perturbation or a cycle-consistency objective would separate those questions more directly. Training for a fixed 60 epochs also gives larger datasets more optimizer updates. Data size and optimization effort are therefore not fully separated.

SCCN confirms that explicit triangle structure is useful. Its clique-complex lift is not representation matched to the graph-based SNNs because it enumerates the target motif. Ridge regression further shows that the sampled GRAPHUNIVERSE regime contains strong graph-size and degree cues. The conclusions may not extend to longer cycles or less structured graph families. They may also differ for higher-order models that are not given the counted motif explicitly.

Acknowledgements

This work was conducted independently, but benefited from discussions at the 2026 London Geometry and Machine Learning (LOGML) Summer School. We are grateful to its participants and organizers for creating a stimulating environment. We also thank Modal for providing free GPU credits.